

MANUAL DE UNITY Y GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para el desarrollo colaborativo de videojuegos en Microsoft Windows con PowerShell 7+, Git Credential Manager, UnityYAMLMerge y GameCI

[Git LFS & Assets Binarios](#)[UnityYAMLMerge 3-Way Driver](#)[Visible Meta Files & GUIDs](#)[Multi-Scene & Prefab Isolation](#)[GameCI & GitHub Actions](#)[Catálogo de Incidentes Críticos](#)

Manual de Unity y GitHub: De Novato a Avanzado en PowerShell (Windows)

Plataforma: Microsoft Windows 10 / Windows 11 / Windows Server

Motor: Unity 2022 LTS / Unity 6 (6000.x)

Shell: PowerShell 7+ (Core) / Windows PowerShell 5.1 (Windows Terminal)

Herramientas: Git for Windows 2.40+, Git LFS, GitHub CLI (`gh`), Git Credential Manager (GCM), Unity Hub, UnityYAMLMerge

Índice de Contenidos

1. [Parte I: Fundamentos y Preparación del Entorno Unity en Windows](#)
 - 1.1 [Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora](#)
 - 1.2 [Instalación de Git, Git LFS y GitHub CLI mediante Winget](#)
 - 1.3 [Configuración Obligatoria del Editor: Visible Meta Files y Force Text](#)
 - 1.4 [La Regla de Oro de los Archivos `.meta` y los GUIDs](#)
 - 1.5 [El Archivo `.gitignore` Oficial y Optimizado para Unity](#)
 - 1.6 [Configuración Exhaustiva de Git LFS con `.gitattributes`](#)
2. [Parte II: Flujo de Trabajo Esencial Diario \(Nivel Novato\)](#)
 - 2.1 [Inicializar y Publicar un Proyecto de Unity desde PowerShell](#)
 - 2.2 [Clonación Correcta de Proyectos con Git LFS en Windows](#)
 - 2.3 [El Ciclo de Trabajo Seguro: Modificar, Probar y Commitear](#)
 - 2.4 [Conventional Commits Aplicados al Desarrollo de Videojuegos](#)
 - 2.5 [Sincronización sin Romper la Cache \(`pull --rebase` \)](#)
3. [Parte III: Estrategias Colaborativas en Unity \(Nivel Intermedio\)](#)
 - 3.1 [Estrategia de Ramas en Equipos de Videojuegos](#)
 - 3.2 [Arquitectura de Escenas Divididas \(Multi-Scene Editing Aditivo\)](#)
 - 3.3 [Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes](#)
 - 3.4 [Configuración de UnityYAMLMerge en Windows como Mergetool](#)
 - 3.5 [Uso de Git Stash y Worktrees sin Desestabilizar Unity](#)
4. [Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente](#)
 - 4.1 [Tema 1: Prevención Arquitectónica de Conflictos en Unity](#)
 - 4.2 [Tema 2: Conflictos en Archivos `.meta` \(GUID Desincronizado\)](#)
 - 4.3 [Tema 3: Conflictos en Scripts C# \(`.cs` \)](#)

- 4.4 [Tema 4: Conflictos en Escenas \(`.unity` \) y Prefabs con UnityYAMLMerge](#)
 - 4.5 [Tema 5: Forzar una Versión Completa de Asset \(`--ours` vs `--theirs` \).](#)
 - 4.6 [Tema 6: Cambios Locales en el Editor al Hacer Pull](#)
 - 4.7 [Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS](#)
 - 4.8 [Tema 8: Conflictos en Archivos Binarios y Bloqueo con Git LFS Lock](#)
 - 4.9 [Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano](#)
5. [Parte V: Herramientas Modernas de Productividad Avanzada](#)
- 5.1 [Git Worktrees en Windows: Trabajar en Múltiples Ramas sin Recargar Unity](#)
 - 5.2 [Depuración Binaria con Git Bisect y Blame en C#](#)
 - 5.3 [GitHub Codespaces desde PowerShell](#)
 - 5.4 [GitHub Copilot CLI en PowerShell](#)
 - 5.5 [Git Hooks y Validación con Pre-commit en Windows](#)
6. [Parte VI: Gestión de Paquetes UPM y Dependencias](#)
- 6.1 [Instalación de Paquetes de Unity Mediante URLs de Git](#)
 - 6.2 [Creación y Publicación de Paquetes UPM en Repositorios Privados](#)
7. [Parte VII: Automatización CI/CD con GitHub Actions y GameCI](#)
- 7.1 [Arquitectura de GameCI para Windows Standalone](#)
 - 7.2 [Activación de Licencias de Unity en GitHub Actions](#)
 - 7.3 [Pipeline Automatizado: Tests y Compilación StandaloneWindows64 \(.exe\)](#)
 - 7.4 [Subida Automática de Instaladores a GitHub Releases](#)
8. [Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos](#)
- 8.1 [Branch Protection Rules y Rulesets](#)
 - 8.2 [Gestión de Secretos para APIs de Juegos \(Steam, Photon, Azure\)](#)
 - 8.3 [Gobernanza con `CODEOWNERS` para Artistas y Programadores](#)
9. [Parte IX: Catálogo Maestro de Incidentes Críticos de Unity en Windows](#)
- 9.1 [Incidente 1: Bloqueo de Archivos por el Proceso de Unity en Windows \(`unlink failed` \).](#)
 - 9.2 [Incidente 2: "Missing Script" Masivo por Desincronización de GUIDs](#)
 - 9.3 [Incidente 3: Escena Corrupta por Conflicto Mal Resuelto](#)
 - 9.4 [Incidente 4: Subida Accidental de la Carpeta `Library/` en Windows](#)
 - 9.5 [Incidente 5: Push Rechazado por Archivo Mayor a 100 MB](#)
 - 9.6 [Incidente 6: Límites de Longitud de Ruta en Windows \(`Filename too long` \).](#)
 - 9.7 [Incidente 7: Desfase de Versiones del Editor de Unity](#)
 - 9.8 [Incidente 8: Límite de Ancho de Banda de Git LFS Superado](#)
 - 9.9 [Incidente 9: Fuga de Claves en ScriptableObjects](#)

- 9.10 [Incidente 10: Limpieza y Reconstrucción Segura de la Cache en Windows](#)
-

Parte I: Fundamentos y Preparación del Entorno

Unity en Windows

1.1 Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora

Un proyecto de Unity en Windows genera gigabytes de archivos de cache y bases de datos SQLite locales que nunca deben enviarse a GitHub:

```
PLAINTEXT

MiJuegoUnity/
├─ Assets/           → [OBLIGATORIO] Scripts C#, Escenas, Prefabs, Modelos, Texturas.
├─ Packages/         → [OBLIGATORIO] manifest.json y packages-lock.json.
├─ ProjectSettings/  → [OBLIGATORIO] Ajustes de física, capas, renderizado y
plataforma.
├─ Library/          → [¡IGNORAR!] Cache gigante compilada por Unity localmente.
├─ Temp/             → [¡IGNORAR!] Archivos temporales de trabajo del editor.
├─ Obj/ & Build/      → [¡IGNORAR!] Binarios y compilaciones ejecutables locales.
├─ UserSettings/     → [¡IGNORAR!] Preferencias y disposición de ventanas del usuario.
└─ Logs/             → [¡IGNORAR!] Registros del editor y compilador.
```

1.2 Instalación de Git, Git LFS y GitHub CLI mediante Winget

Abre **PowerShell** como Administrador:

```
POWERSHELL

winget install --id Git.Git -e --source winget
winget install --id GitHub.GitLFS -e --source winget
winget install --id GitHub.cli -e --source winget
git lfs install
```

¿Qué hace este comando?

Instala los binarios oficiales para Windows de Git, el soporte de archivos pesados Git LFS y GitHub CLI, e inicializa los filtros globales de Git LFS en tu sistema.

1.3 Configuración Obligatoria del Editor: Visible Meta Files y Force Text

Antes de hacer el primer commit:

1. Abre tu proyecto en el Editor de Unity.
2. Ve a **Edit → Project Settings → Version Control**.
 - Configura **Mode** en: `Visible Meta Files`.
3. Ve a **Edit → Project Settings → Editor**.
 - Configura **Asset Serialization Mode** en: `Force Text`.

IMPORTANT

`Force Text` obliga a Unity a escribir las escenas (`.unity`) y prefabs (`.prefab`) en formato de texto plano YAML. Esto permite que Git pueda comparar versiones y resolver discrepancias semánticamente.

1.4 La Regla de Oro de los Archivos `.meta` y los GUIDs

Cada archivo dentro de `Assets/` posee un archivo `.meta` con un identificador GUID único:

- Si creas `Enemigo.cs`, Unity crea `Enemigo.cs.meta`.
- Si mueves, renombras o eliminas `Enemigo.cs` desde el Explorador de Windows, **debes mover, renombrar o eliminar** `Enemigo.cs.meta` exactamente igual.

CAUTION

Si subes un asset a GitHub sin su archivo `.meta`, el Unity de tu compañero creará un `.meta` nuevo con un GUID aleatorio diferente, provocando el fallo masivo de componentes desvinculados (*Missing Script y Missing Prefab*).

1.5 El Archivo `.gitignore` Oficial y Optimizado para Unity

Crea `.gitignore` en la raíz de tu proyecto de Unity:

```
/[Ll]ibrary/  
/[Tt]emp/  
/[Oo]bj/  
/[Bb]uild/  
/[Bb]uilds/  
/[Ll]ogs/  
/[Uu]ser[Ss]ettings/  
/[Mm]emoryCaptures/  
  
# Visual Studio & JetBrains Rider  
.vs/  
*.csproj  
*.unityproj  
*.sln  
*.suo  
*.user  
*.userprefs  
*.pidb  
*.booproj  
*.pdb  
*.opendb  
  
# Compilaciones  
*.apk  
*.aab  
*.unitypackage  
*.exe  
  
# S0  
Thumbs.db  
desktop.ini  
$RECYCLE.BIN/  
.DS_Store
```

1.6 Configuración Exhaustiva de Git LFS con `.gitattributes`

Crea `.gitattributes` en la raíz del proyecto para gestionar archivos binarios pesados con Git LFS y fijar finales de línea LF para evitar diferencias artificiales en Windows:

```
# Finales de línea normalizados
* text=auto eol=lf
*.cs text diff=csharp
*.shader text
*.hlsl text
*.compute text
*.json text
*.yaml text

# Integración con UnityYAMLMerge
*.unity merge=unityyamlmerge eol=lf
*.prefab merge=unityyamlmerge eol=lf
*.asset merge=unityyamlmerge eol=lf
*.mat merge=unityyamlmerge eol=lf

# Modelos 3D en Git LFS
*.fbx filter=lfs diff=lfs merge=lfs -text
*.obj filter=lfs diff=lfs merge=lfs -text
*.blend filter=lfs diff=lfs merge=lfs -text
*.max filter=lfs diff=lfs merge=lfs -text

# Texturas y Gráficos en Git LFS
*.png filter=lfs diff=lfs merge=lfs -text
*.jpg filter=lfs diff=lfs merge=lfs -text
*.tga filter=lfs diff=lfs merge=lfs -text
*.psd filter=lfs diff=lfs merge=lfs -text
*.exr filter=lfs diff=lfs merge=lfs -text
*.hdr filter=lfs diff=lfs merge=lfs -text

# Audio y Video en Git LFS
*.wav filter=lfs diff=lfs merge=lfs -text
*.mp3 filter=lfs diff=lfs merge=lfs -text
*.ogg filter=lfs diff=lfs merge=lfs -text
*.mp4 filter=lfs diff=lfs merge=lfs -text

# Binarios y librerías
*.dll filter=lfs diff=lfs merge=lfs -text
*.so filter=lfs diff=lfs merge=lfs -text
```

Parte II: Flujo de Trabajo Esencial Diario (Nivel Novato)

2.1 Inicializar y Publicar un Proyecto de Unity desde PowerShell

```
Set-Location "C:\UnityProjects\MiVideojuego"
git init
git lfs install
git add .gitignore .gitattributes
git commit -m "chore: configurar gitignore y lfs para Unity"
git add Assets/ Packages/ ProjectSettings/
git commit -m "feat: inicializar estructura principal del videojuego"
gh repo create MiVideojuego --public --source=. --remote=origin --push
```

¿Qué hace este comando?

Inicializa Git, configura los filtros LFS, añade los archivos de configuración base y sube las 3 carpetas del proyecto a un nuevo repositorio en GitHub.

2.2 Clonación Correcta de Proyectos con Git LFS en Windows

```
git clone git@github.com:usuario/MiVideojuego.git
Set-Location .\MiVideojuego
git lfs pull
```

¿Qué hace este comando?

Descarga el repositorio y fuerza la descarga completa de los archivos binarios reales gestionados por LFS.

2.3 El Ciclo de Trabajo Seguro: Modificar, Probar y Commitear

1. En Unity, guarda la escena (`Ctrl + S`).

2. Espera a que termine la compilación de scripts en Unity.
3. En PowerShell:

```
git status -s
git add Assets/Scripts/Player.cs Assets/Scripts/Player.cs.meta
git commit -m "feat(player): agregar control de salto con inercia"
```

¿Qué hace este comando?

Revisa los cambios y prepara de forma inseparable el script y su archivo `.meta`.

2.4 Conventional Commits Aplicados al Desarrollo de Videojuegos

```
git commit -m "feat(ui): implementar barra de estamina dinámica"
git commit -m "fix(physics): corregir colisión con rampas pronunciadas"
git commit -m "art(environment): importar modelo 3D de árbol con LODs"
```

2.5 Sincronización sin Romper la Cache (`pull --rebase`)

```
git fetch origin
git pull --rebase origin main
git push origin main
```

¿Qué hace este comando?

Descarga los commits del equipo y sitúa tus commits locales en la cima de forma lineal. Unity reimportará limpiamente solo los archivos alterados.

Parte III: Estrategias Colaborativas en Unity (Nivel Intermedio)

3.1 Estrategia de Ramas en Equipos de Videojuegos

```
git switch -c feature/sistema-dialogos
git push -u origin feature/sistema-dialogos
```

3.2 Arquitectura de Escenas Divididas (Multi-Scene Editing Aditivo)

No trabajen todos sobre la misma escena. Dividan cada nivel en sub-escenas:

- Nivel_01_Logica.unity (GameManagers, Spawners).
- Nivel_01_Arte.unity (Modelos, luces, mallas de colisión).
- Nivel_01_Audio.unity (Efectos de sonido y emisores 3D).

Cárguenlas en tiempo de ejecución con:

```
SceneManager.LoadSceneAsync("Nivel_01_Arte", LoadSceneMode.Additive);
```

3.3 Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes

Encapsula personajes, trampas e interfaces dentro de **Prefabs**. Edita los elementos en el modo de aislamiento de Prefabs para que los cambios se guarden en `.prefab` sin tocar el archivo `.unity` de la escena principal.

3.4 Configuración de UnityYAMLMerge en Windows como Mergetool

Localiza tu instalación de Unity (por ejemplo: `C:\Program Files\Unity\Hub\Editor\2022.3.20f1\Editor\Data\Tools\UnityYAMLMerge.exe`).

Configúralo en PowerShell:

```
POWERSHELL

$unityMerge = 'C:\Program
Files\Unity\Hub\Editor\2022.3.20f1\Editor\Data\Tools\UnityYAMLMerge.exe'

git config --global merge.unityyamlmerge.name "Unity SmartMerge"
git config --global merge.unityyamlmerge.driver "`"$unityMerge`" merge -h -p --mode=落后 %0
%A %B %A"
git config --global merge.unityyamlmerge.recursive binary
```

¿Qué hace este comando?

Enlaza el solucionador semántico nativo de Unity a Git para resolver automáticamente colisiones de GameObjects en escenas y prefabs.

3.5 Uso de Git Stash y Worktrees sin Desestabilizar Unity

```
POWERSHELL

# Crear un worktree en una carpeta paralela para no perder la cache de Library
git worktree add ..\MiJuego-Bugfix hotfix/correccion-urgente
```

¿Qué hace este comando?

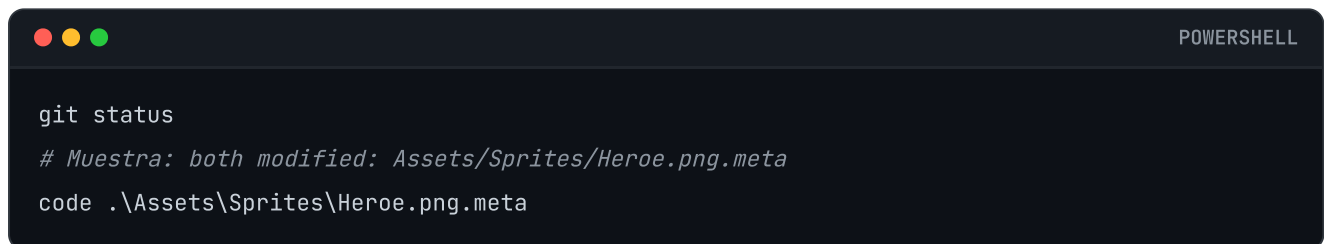
Extrae una rama en una carpeta física paralela. Puedes abrir una segunda ventana de Unity sin vaciar la cache de tu proyecto principal.

Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente

4.1 Tema 1: Prevención Arquitectónica de Conflictos en Unity

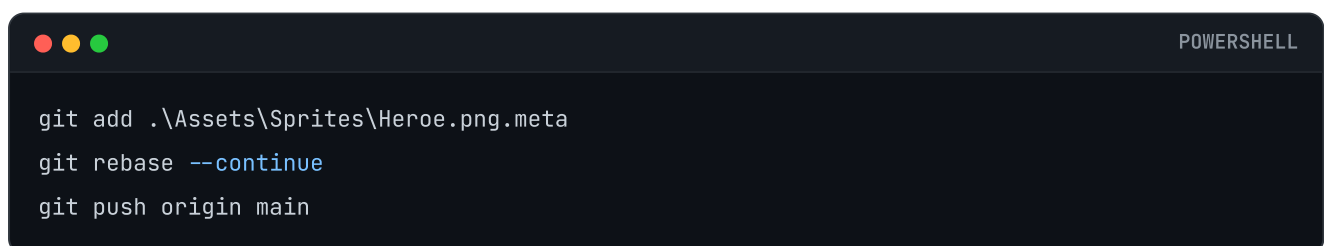
1. **Escenas Aditivas:** Dividir niveles en sub-escenas independientes.
2. **Uso riguroso de Prefabs:** Toda entidad interactiva es un Prefab.
3. **Bloqueo exclusivo:** Usar `git lfs lock` para escenas críticas.

4.2 Tema 2: Conflictos en Archivos `.meta` (GUID Desincronizado)



```
git status
# Muestra: both modified: Assets/Sprites/Heroe.png.meta
code .\Assets\Sprites\Heroe.png.meta
```

1. Conserva un único valor en la clave `guid:`. Elige el GUID que ya estaba referenciado en escenas existentes.
2. Elimina los marcadores `<<<<<<`, `=====` y `>>>>>>`.
3. Guarda y ejecuta:



```
git add .\Assets\Sprites\Heroe.png.meta
git rebase --continue
git push origin main
```

4.3 Tema 3: Conflictos en Scripts C# (.cs)

```
git status
code .\Assets\Scripts\Player.cs
git add .\Assets\Scripts\Player.cs
git rebase --continue
git push origin main
```

POWERSHELL

4.4 Tema 4: Conflictos en Escenas (.unity) y Prefabs con UnityYAMLMerge

```
git mergetool
```

POWERSHELL

¿Qué hace este comando?

Ejecuta `UnityYAMLMerge`. Si los `GameObjects` no colisionan en la misma propiedad, Unity reconciliará la escena automáticamente.

Si requiere intervención manual, abrirá VS Code. Revisa, guarda y ejecuta:

```
git add .\Assets\Scenes\Nivel1.unity
git commit -m "merge: resolver colisión en escena Nivel1"
git push origin main
```

POWERSHELL

4.5 Tema 5: Forzar una Versión Completa de Asset (--ours vs --theirs)

```
# Conservar mi versión local completa (escena y meta):
git checkout --ours .\Assets\Scenes\Nivel1.unity .\Assets\Scenes\Nivel1.unity.meta
git add .\Assets\Scenes\Nivel1.unity .\Assets\Scenes\Nivel1.unity.meta
git commit -m "resolve: conservar versión local de la escena"
git push origin main
```

POWERSHELL

```
# Aceptar la versión remota del compañero completa:
git checkout --theirs .\Assets\Scenes\Nivel1.unity .\Assets\Scenes\Nivel1.unity.meta
git add .\Assets\Scenes\Nivel1.unity .\Assets\Scenes\Nivel1.unity.meta
git commit -m "resolve: aceptar versión remota del compañero"
git push origin main
```

4.6 Tema 6: Cambios Locales en el Editor al Hacer Pull

```
git stash save "Ajustes locales del inspector"
git pull --rebase origin main
git stash pop
```

4.7 Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS

```
git pull --rebase origin main
git lfs push --all origin main
git push origin main
```

4.8 Tema 8: Conflictos en Archivos Binarios y Bloqueo con Git LFS Lock

```
# Bloquear un modelo 3D antes de empezar a trabajar en él
git lfs lock Assets/Modelos/Heroe.fbx

# Comprobar bloqueos activos en el repositorio
git lfs locks

# Liberar el bloqueo al terminar y subir cambios
git lfs unlock Assets/Modelos/Heroe.fbx
```

4.9 Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano



```
# Para mantener el asset:
git add .\Assets\Scripts\Bala.cs .\Assets\Scripts\Bala.cs.meta
git commit -m "resolve: conservar script y meta"

# Para confirmar el borrado:
git rm .\Assets\Scripts\Bala.cs .\Assets\Scripts\Bala.cs.meta
git commit -m "resolve: confirmar borrado de script y meta"
```

Parte V: Herramientas Modernas de Productividad Avanzada

5.1 Git Worktrees en Windows: Trabajar en Múltiples Ramas sin Recargar Unity

```
git worktree add ..\MiJuego-Test feature/pruebas-fisica
Set-Location ..\MiJuego-Test
git commit -am "test: ajustar gravedad de proyectiles"
git push origin feature/pruebas-fisica

Set-Location ..\MiJuego
git worktree remove ..\MiJuego-Test
```

5.2 Depuración Binaria con Git Bisect y Blame en C#

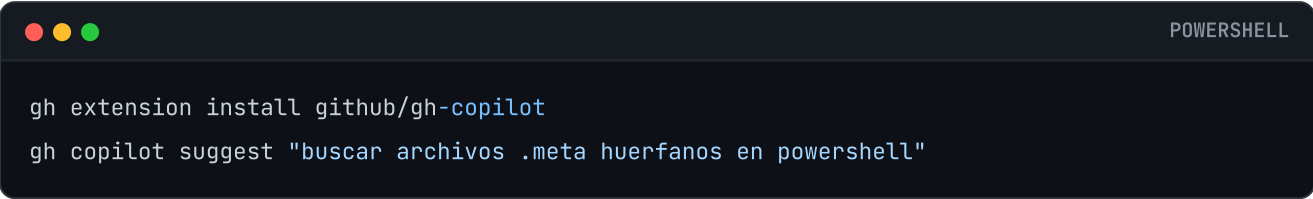
```
git bisect start
git bisect bad
git bisect good v1.0.0

# Probar compilación en Unity y marcar:
git bisect good # 0: git bisect bad
git bisect reset
```

5.3 GitHub Codespaces desde PowerShell

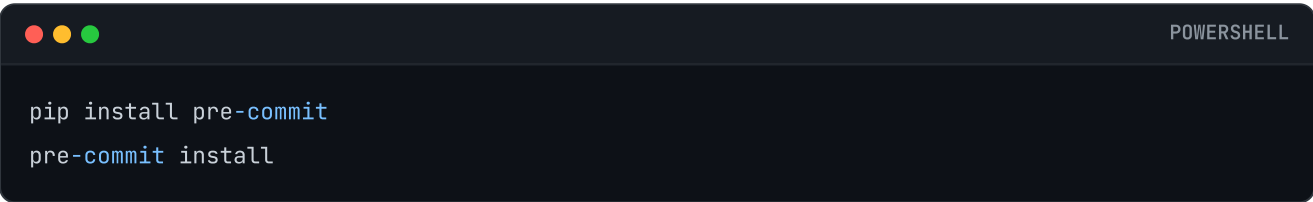
```
gh codespace create --repo usuario/MiVideojuego --branch main
gh codespace code -c nombre-del-codespace
```

5.4 GitHub Copilot CLI en PowerShell



```
gh extension install github/gh-copilot
gh copilot suggest "buscar archivos .meta huérfanos en powershell"
```

5.5 Git Hooks y Validación con Pre-commit en Windows



```
pip install pre-commit
pre-commit install
```

Parte VI: Gestión de Paquetes UPM y Dependencias

6.1 Instalación de Paquetes de Unity Mediante URLs de Git

Edita `Packages/manifest.json` :

A screenshot of a code editor window with a dark theme. The title bar shows three colored window control buttons (red, yellow, green) on the left and the text 'JSON' on the right. The editor contains the following JSON code:

```
{
  "dependencies": {
    "com.cysharp.unitask": "https://github.com/Cysharp/UniTask.git?
path=src/UniTask/Assets/Plugins/UniTask#2.5.5"
  }
}
```

6.2 Creación y Publicación de Paquetes UPM en Repositorios Privados

Estructura de paquetes modulares independientes con su propio `package.json` .

Parte VII: Automatización CI/CD con GitHub Actions y GameCI

7.1 Arquitectura de GameCI para Windows Standalone

GameCI ejecuta compilaciones headless de Unity utilizando licencias activadas mediante variables de entorno secretas.

7.2 Activación de Licencias de Unity en GitHub Actions

Configura en los secretos del repositorio: `UNITY_EMAIL`, `UNITY_PASSWORD`, `UNITY_LICENSE`.

7.3 Pipeline Automatizado: Tests y Compilación StandaloneWindows64 (.exe)

Crea `.github/workflows/unity-windows-ci.yml`:

```
name: Unity Windows Build CI

on:
  push:
    branches: [ main ]

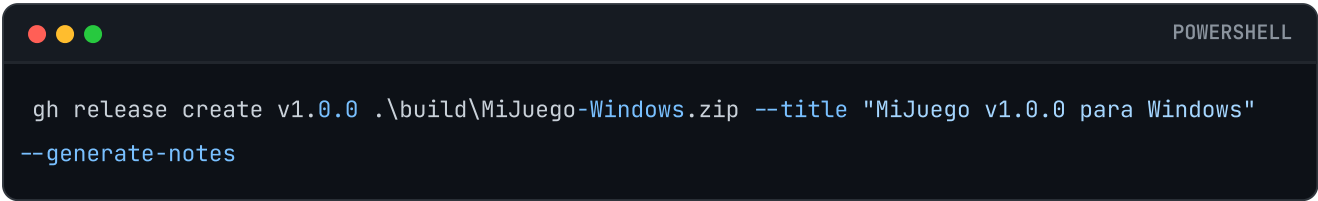
jobs:
  build-windows:
    name: Compilar StandaloneWindows64
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: true

      - uses: actions/cache@v4
        with:
          path: Library
          key: Library-Windows-${{ hashFiles('Assets/**', 'Packages/**',
'ProjectSettings/**') }}

      - uses: game-ci/unity-builder@v4
        env:
          UNITY_LICENSE: ${ secrets.UNITY_LICENSE }}
          UNITY_EMAIL: ${ secrets.UNITY_EMAIL }}
          UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }}
        with:
          projectPath: .
          targetPlatform: StandaloneWindows64
          buildName: MiJuegoWindows
          buildsPath: build/Windows

      - uses: actions/upload-artifact@v4
        with:
          name: MiJuego-Windows-x64
          path: build/Windows/StandaloneWindows64
```

7.4 Subida Automática de Instaladores a GitHub Releases



A screenshot of a Windows PowerShell terminal window. The window has a dark background and a title bar with three colored buttons (red, yellow, green) on the left and the word "POWERSHELL" on the right. The terminal displays a single command: `gh release create v1.0.0 .\build\MiJuego-Windows.zip --title "MiJuego v1.0.0 para Windows" --generate-notes`. The command is split across two lines: the first line contains `gh release create v1.0.0 .\build\MiJuego-Windows.zip --title "MiJuego v1.0.0 para Windows"` and the second line contains `--generate-notes`.

```
gh release create v1.0.0 .\build\MiJuego-Windows.zip --title "MiJuego v1.0.0 para Windows"
--generate-notes
```

Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos

8.1 Branch Protection Rules y Rulesets

- Prohibir commits directos a `main`.
 - Requerir que los tests automatizados de GameCI terminen en verde.
 - Bloquear force-pushes (`git push --force`).
-

8.2 Gestión de Secretos para APIs de Juegos (Steam, Photon, Azure)

Nunca incluyas API keys privadas de Steamworks o Azure en scripts de C# en el repositorio. Inyéctalas en tiempo de compilación o mediante variables de entorno locales.

8.3 Gobernanza con CODEOWNERS para Artistas y Programadores

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text is in a light gray monospace font. In the top-right corner, the word "PLAINTEXT" is displayed. The content of the terminal is a CODEOWNERS file configuration.

```
# .github/CODEOWNERS
/Assets/Scripts/ @estudio/programadores
/Assets/Modelos/ @estudio/artistas-3d
/ProjectSettings/ @estudio/lead-developer
```

Parte IX: Catálogo Maestro de Incidentes Críticos de Unity en Windows

9.1 Incidente 1: Bloqueo de Archivos por el Proceso de Unity en Windows (unlink failed)

Problema: Al hacer `git switch` o `git pull`, Git falla con el error:

error: unable to unlink old 'Library/...': Permission denied o unlink of file failed. Should I try again? .

Causa: El Editor de Unity mantiene abiertos ficheros de bases de datos internas PDB o SQLite.

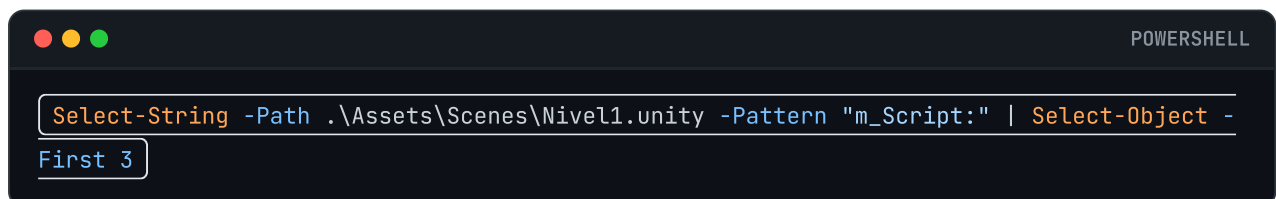


```
POWERSHELL

# Cerrar limpiamente el proceso de Unity desde PowerShell antes de reintentar
Get-Process Unity -ErrorAction SilentlyContinue | Stop-Process -Force
git checkout -f rama-destino
```

9.2 Incidente 2: "Missing Script" Masivo por Desincronización de GUIDs

1. Abre la escena en VS Code y localiza el GUID faltante:



```
POWERSHELL

Select-String -Path .\Assets\Scenes\Nivel1.unity -Pattern "m_Script:" | Select-Object -
First 3
```

2. Restaura ese GUID exacto en el archivo `.meta` del script correspondiente.

9.3 Incidente 3: Escena Corrupta por Conflicto Mal Resuelto

```
# Localizar delimitadores de conflicto residuales
Select-String -Path .\Assets\Scenes\Nivel1.unity -Pattern "^(<<<<<<|=====|>>>>>>)"
```

Elimina los marcadores en VS Code y guarda el archivo.

9.4 Incidente 4: Subida Accidental de la Carpeta `Library/` en Windows

```
git rm -r --cached Library/
Add-Content -Path ".gitignore" -Value "`nLibrary/"
git commit -m "chore: remover Library del seguimiento"
pip install git-filter-repo
git filter-repo --path Library --invert-paths --force
git push origin --force --all
```

9.5 Incidente 5: Push Rechazado por Archivo Mayor a 100 MB

```
git lfs migrate import --include="*.fbx,*.wav,*.mp4,*.psd" --everything
git push origin --force --all
```

9.6 Incidente 6: Límites de Longitud de Ruta en Windows (`Filename too long`)

```
git config --system core.longpaths true
```

9.7 Incidente 7: Desfase de Versiones del Editor de Unity

Comprueba `ProjectSettings/ProjectVersion.txt` para asegurar que todo el equipo utilice exactamente la misma versión del motor.

9.8 Incidente 8: Límite de Ancho de Banda de Git LFS Superado

Amplía la cuota de LFS en GitHub o configura un endpoint LFS privado.

9.9 Incidente 9: Fuga de Claves en ScriptableObjects

```
git filter-repo --path Assets/Resources/ConfiguracionSecreta.asset --invert-paths --force
git push origin --force --all
```

9.10 Incidente 10: Limpieza y Reconstrucción Segura de la Cache en Windows

```
Get-Process Unity -ErrorAction SilentlyContinue | Stop-Process -Force
Remove-Item -Recurse -Force .\Library, .\Temp, .\Obj
```

¿Qué hace este comando?

Elimina la cache local corrupta. Al abrir el proyecto en Unity Hub, el motor reconstruirá automáticamente todos los assets de forma limpia.